



HIGH PERFORMANCE COMPUTING

Rapport TP

Élèves :

Aïssa PANSAN

S9253377

Enseignant :

Danielle D'AGOSTINO

29 Mars 2025

1 Contexte

Le but de ce TP est d'étudier les concepts vu en cours notamment sur l'influence de la vectorisation sur les performances du code (FLOPS). On va utiliser le logiciel **Intel Advisor** ainsi que le **HPC Toolkit Pack** d'Intel pour ce faire.

2 Installation de l'environnement

Pour pouvoir faire le TP, il est nécessaire d'avoir l'environnement de développement d'Intel, ce dernier comprends :

- **Un compilateur fait par Intel icpx**
- **Librairies MPI**
- **Intel Advisor**
- Et d'autre encore (Principalement des dépendances au truc précédents)

On peut obtenir tous ces éléments via le **oneAPI HPC Toolkit** ou le **oneAPI Base Toolkit**. Le seul hic est la taille des toolkit (**Plus de 10Go, en enlevant les librairies Data Analysis**). J'espère que tu as de l'espace.

Les étapes pour avoir ton environnement complet sont les suivantes :

1. Installer l'un des deux Toolkit.
2. Lancer le .sh (En oubliant pas de rendre le script executable chmod)
3. Suivre les instructions. Tu peux laisser tomber la selection de l'IDE, ça marche pas (pour le moment)
4. Si tu es un peu limiter en espace, selectionne ses librairies.
5. Une fois que tout est installer, lance dans un shell l'initialisation des variable de chemin (Action uniquement valable sur ton shell) : **source /Program_externe/intel/oneapi/...**
6. Tu peux enfin lancer Advisor : **advisor-gui**. (Sauf, si tu l'a installer autre part.

Dans ton cas, lance Advisor depuis cette endroit : `/opt/intel/oneapi/advisor/2025.3/bin64/advisor-gui`

3 Initialisation du programme

Le programme C++ qu'on va lancer se définit en 4 groupes de calculs, plus ou moins complexe. Chaque groupes possède aussi plusieurs déclinaisons (AOS = Array of Struct, SOA = Struct of Array,...). Ces déclinaison permettent de mettre en évidence d'utiliser dans certains le CPU de manière plus optimale, notamment avec le cache).

Avant de commencer on doit procéder à quelque paramétrages :

- Ajouter les flags suivant dans CXXFLAGS dans le Makefile : `-diag-disable=10441` et `-march=alderlake`
- Dans **default** ajouter **-diag-disable=10441 -qopenmp** après `$(CXX)` (le dernier point ne semble pas obligatoire, je l'ai mis après `rooffline_demo`).
- **Spécifique à mon architecture** : Changer le compilateur par **icpx** par défaut, c'est `icpc`. Remarque : On change de compilateur, car dans les machines du labo, il utilise l'ancien compilateur d'Intel `icpc`. Ce dernier n'est plus supporté par Intel depuis 2022. Il utilise désormais `icpx`. Vu que tu as installer le Toolkit sur Internet, tu dispose du dernier compilateur d'Intel. Cela va poser quelque soucis avec l'exemple **rooffline.cpp** qui lui est rester dans l'ancienne version du compilateur.

Avant de lancer Make, il faut modifier le fichier `rooffline.cpp`. dans `src`.

- Mettre en commentaire toutes les occurences de **pragma unroll (UNROLL_COUNT)**
- Supprimer toutes les occurences de **simdlen(VECTOR_LENGTH)**. (tips dans vim utilise la commande `replace :%s/simdlen(VECTOR_LENGTH)/`
- **Spécifique à mon architecture** : Par défaut dans le code ligne 292 et 293. La déclaration de deux `pragma` à la suite ne rend pas très heureux le compilateur qui s'attend plutôt à une boucle juste après **pragma nonroll**. La solution pour contourner ce problème est de déclarer les deux instructions à la suite : **pragma nounroll omp simd**. Tu obtiendra un Warning, mais c'est mieux que rien.
- Tu peux enfin **Make**.
- Lance le programme **rooffline_demo** dans `rooffline_demo_sample/release`

4 Paramétrisation de Intel Advisor

Promis, on arrêtera de paramétrer quoi ce soit, si ce n'est que `roofline.cpp` après. Dans Intel Advisor

- Génère un projet, dans le paramètre **Application** sélectionne **roofline_demo**.

Le **Working Directory** est le dossier **release**

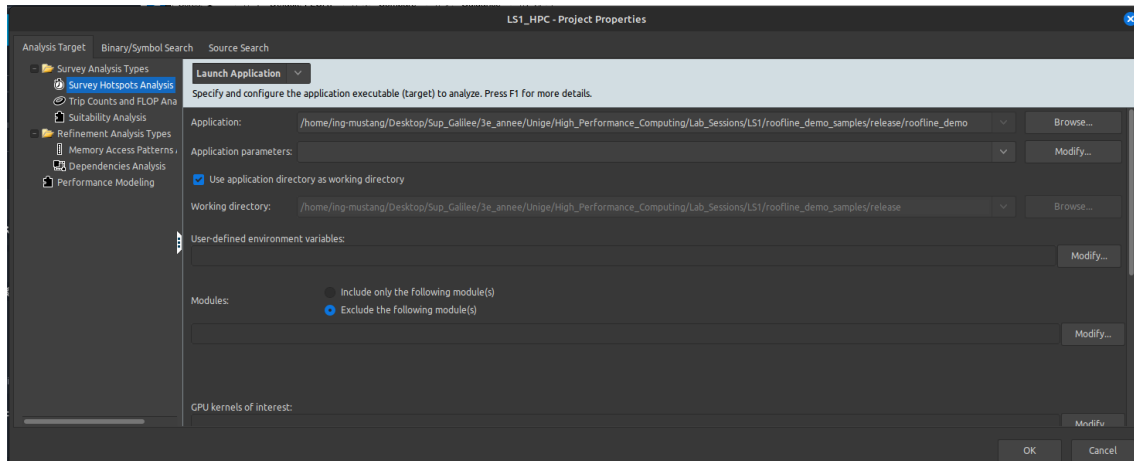


FIGURE 1 – Configuration

- Dans la section **Binary/Symbol Search**, renseigne les dossiers **release** et **src/**

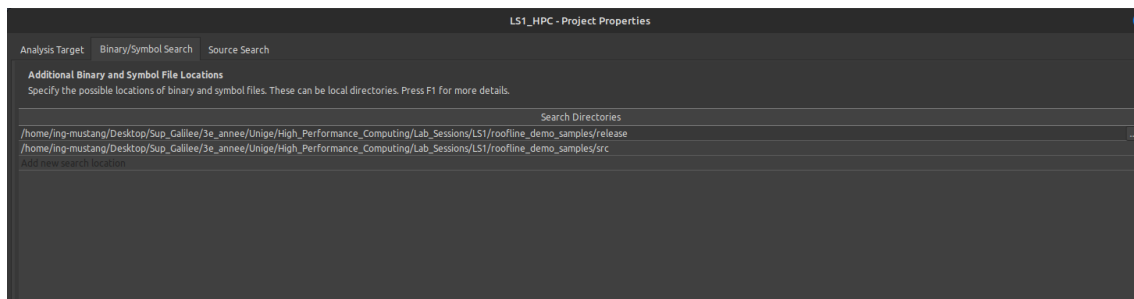
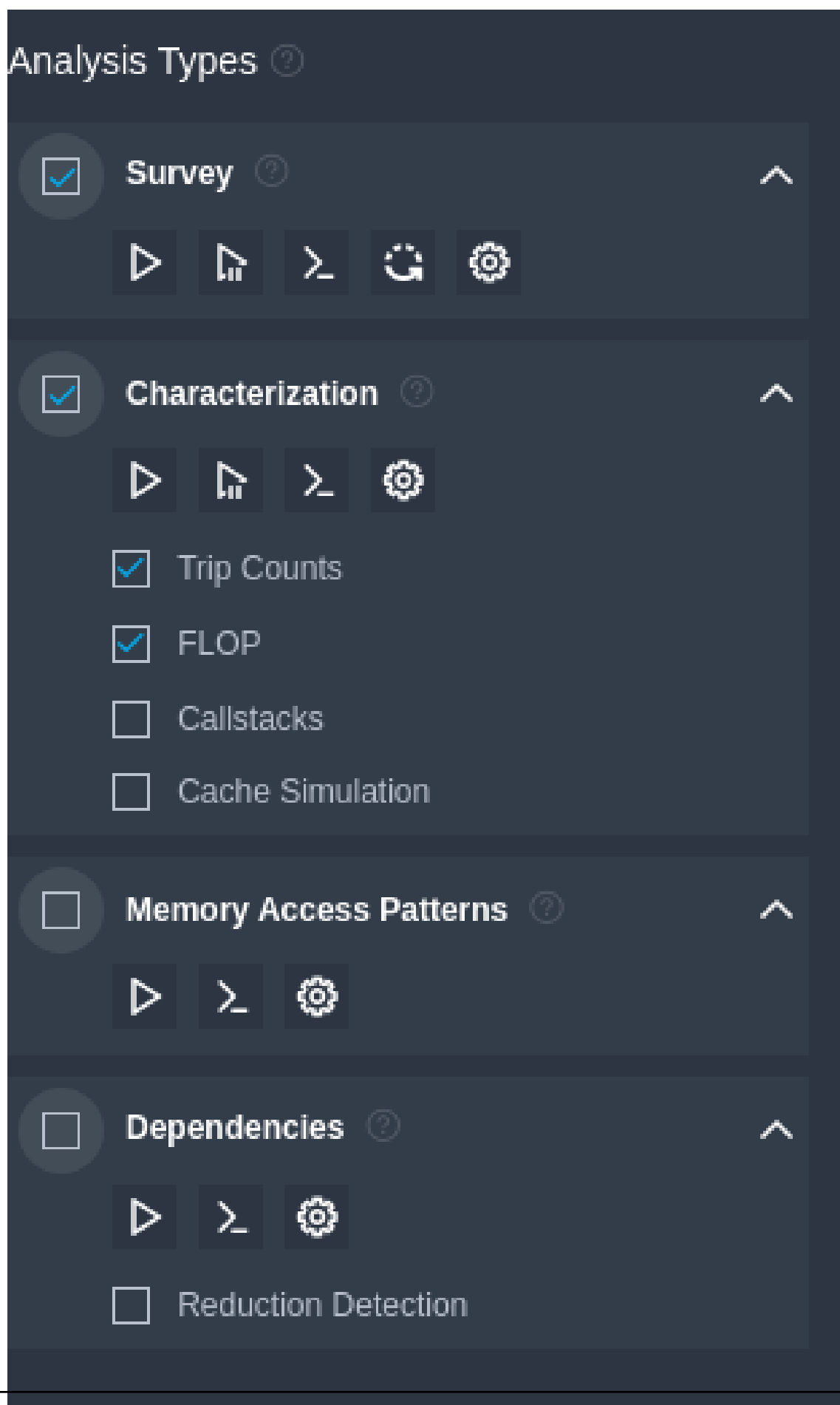


FIGURE 2 – Configuration

- Tu peux enfin, lancer ta simulation, en sélectionnant les 3 paramètres défini dans l'image ci-dessous.



5 Interpreter les résultats

À chaque fois, qu'on veut lancer un groupe précis, on doit mettre en commentaire, les autres groupes et leur déclinaisons. Ainsi que les déclinaisons de notre propre groupe, sauf 1, bien sur (Sinon on lance rien). Ne pas oublier de lancer make à chaque fois qu'on change de groupe/déclinaisons

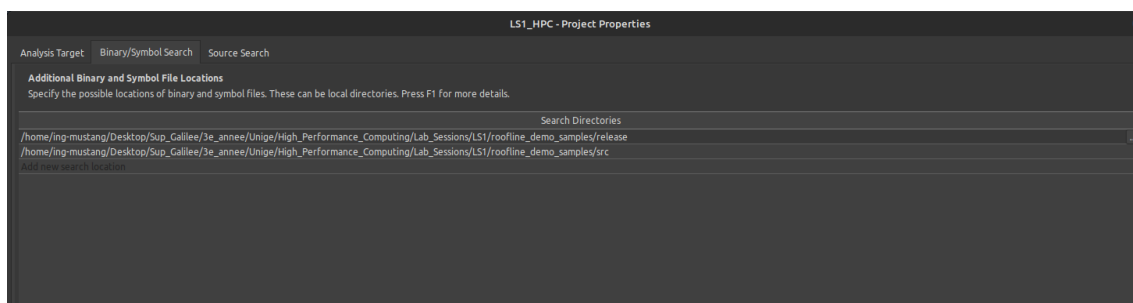


FIGURE 4 – On s'apprete à lancer le Groupe 1 - Type SOA Scalar

5.1 Groupe 1 - Type SOA Scalar

Quand on lance la simulation, on voit le modèle roofline associé à notre simulation. Ici, on voit